DOCUMENTO TÉCNICO

FrescApp - Back-end del Hito 2

Asignatura: INGT1003 · Arquitectura de Desarrollo (Móvil y Web)

Proyecto: FrescApp · **Versión de API:** v1

Universidad Mayor · Escuela de Ingeniería Civil

Docente: Prof. Carlos Muñoz S. · **Equipo:** FrescApp · **Fecha:** 4 de noviembre de 2026

Contenido

1. Introducción
2. Alcance y trazabilidad con el Hito 1
    2.1 Cambios respecto al Hito 1
3. Tecnologías y decisión arquitectónica
4. Capas y recorrido de una petición
5. Contrato REST
6. Persistencia y credenciales
7. Instalación y demostración local
8. Conclusión
9. Referencias técnicas

1. Introducción

Este documento describe el servicio de back-end preparado para FrescApp en el Hito 2. Presenta la API REST versionada, su organización por capas, el modelo de datos en Firestore y la forma de levantar una demostración local sin credenciales privadas. El servicio conserva los datos y la autenticación existentes, y permite demostrar las operaciones CRUD para inventario y lista de compras.

2. Alcance y trazabilidad con el Hito 1

El Hito 1 planteó una aplicación móvil y web para registrar alimentos del hogar, consultar sus vencimientos y mantener datos en Firebase. En esta etapa se agrega una interfaz HTTP REST propia en Cloud Functions. La API mantiene Firestore como motor de persistencia y Firebase Authentication como identidad; no incorpora catálogos comerciales ni otros servicios de datos de terceros.

Cambios respecto al Hito 1

Qué cambió	Por qué	Impacto y alcance
Se agregó una API REST propia y versionada (`/v1`) en Cloud Functions para inventario y lista de compras.	Acordar rutas, métodos, parámetros, cuerpos y errores que pueda consumir cualquier cliente HTTP.	El contrato se puede demostrar con Postman. No se agregan APIs externas ni se reemplaza Firebase.
La API verifica el ID token, obtiene de él el UID y valida las entradas en el servidor antes de acceder a Firestore con Admin SDK.	Aplicar identidad, propiedad de los datos y reglas de dominio en el servidor; Admin SDK no aplica las reglas cliente de Firestore.	El backend asume esas verificaciones y el manejo de errores. Firebase Authentication sigue siendo la identidad y Firestore conserva los datos.
Las rutas nuevas usan `usuarios/{uid}/inventario` y `usuarios/{uid}/listaCompras`, las subcolecciones por usuario ya usadas por la aplicación.	Mantener la separación de datos por cuenta y reutilizar el modelo existente.	No requiere migrar esos registros. El inventario histórico en `inventario/{id}` queda fuera de estas rutas y sigue atendido por el flujo anterior de la app.
Las pantallas existentes todavía acceden directamente a Firestore; no se conectaron a la nueva API en este hito.	Evaluar y entregar el backend de forma independiente del cambio de cliente.	La API se demuestra con Postman. Integrar las pantallas y retirar su acceso directo a Firestore queda para una etapa posterior.

3. Tecnologías y decisión arquitectónica

Para elegir la arquitectura separamos dos decisiones: la plataforma que ejecuta y publica el servicio, y el framework que organiza las rutas HTTP dentro de esa plataforma. Cloud Functions y Cloud Run resuelven el primer problema; Express y Fastify resuelven el segundo.

Plataforma de ejecución

Plataforma	Qué resuelve	Ventajas y costos para FrescApp
Cloud Functions v2 con `onRequest`	Publica una función como handler HTTPS y administra su ejecución bajo demanda.	Se integra con Firebase Authentication y Firestore Admin y reutiliza la configuración existente. El costo es depender del entorno Firebase y habilitar facturación para producción.
Cloud Run	Ejecuta una aplicación o contenedor como servicio administrado.	Da control sobre el proceso y permite elegir el servidor HTTP. El costo es construir y desplegar un contenedor y operar un servicio adicional con su configuración y monitoreo (Google Cloud, s. f.).

Framework HTTP

Opción	Qué resuelve	Ventajas y costos para FrescApp
Router propio actual	`functions/api/routes.js` asocia cada método y ruta con una operación.	Es pequeño y transparente para los dos recursos del Hito 2 y no agrega una dependencia. El costo es mantener manualmente el despacho de rutas, la composición de middleware y el formato de errores.
Express	Organiza endpoints por método y ruta y encadena middleware para procesar solicitudes y respuestas.	Tiene un modelo de rutas y middleware ampliamente usado; incluye parser JSON y permite definir middleware de error. El costo es agregar una dependencia y todavía definir por separado las reglas de validación del dominio (Express.js, s. f.-a, s. f.-b).
Fastify	Organiza rutas y plugins y permite declarar esquemas para validar solicitudes y serializar respuestas.	Los esquemas JSON pueden hacer más explícito el contrato HTTP. El costo es definir y mantener esos esquemas y adoptar sus convenciones (Fastify, s. f.).

Express y Fastify pueden ejecutarse dentro de Cloud Functions o Cloud Run: son opciones para la capa HTTP, no plataformas de despliegue.

En FrescApp elegimos Cloud Functions v2 como plataforma, `onRequest` como entrada HTTPS y el router propio como enrutador. La API tiene dos recursos y un conjunto acotado de rutas; sus validaciones y respuestas se implementan explícitamente en los módulos existentes. Por ahora, la composición de middleware de Express o los esquemas de Fastify no justifican otra dependencia. Aceptamos a cambio mantener nuestro propio despacho de rutas y manejo de errores. Si aumentan los recursos o las rutas transversales de autenticación, registro y validación, Express o Fastify podrían resolver esa repetición.

Elemento	Elección	Motivo y costo asumido
Lenguaje	JavaScript en Node.js 22	Reutiliza el runtime ya usado por Functions y permite compartir el entorno del proyecto. Se asume tipado dinámico y se compensa con validación explícita de entradas.
Plataforma y entrada HTTP	Firebase Cloud Functions v2 (`onRequest`)	Proporciona HTTPS, ejecución bajo demanda y despliegue integrado con Firebase (Firebase, s. f.-c, s. f.-d). Se asume dependencia operativa de Firebase y disponibilidad del plan requerido para producción; el emulador permite desarrollo local.
Enrutamiento HTTP	Router propio en `functions/api/routes.js`	La tabla de rutas se mantiene simple para dos recursos y no suma un framework. El costo es mantener manualmente las convenciones que Express o Fastify ofrecen.
Persistencia	Cloud Firestore con Firebase Admin SDK	Mantiene el modelo documental y los datos ya usados por FrescApp; el Admin SDK permite escribir documentos desde un entorno confiable (Firebase, s. f.-a, s. f.-b). Se asumen costos por lecturas/escrituras y límites de consulta; la API pagina colecciones hasta 100 registros por petición.
Autenticación	Firebase Authentication ID tokens	El servidor verifica el token y obtiene el UID del usuario con el Admin SDK (Firebase, s. f.-e). El servidor no acepta un UID enviado por el cliente.

Monolito modular frente a microservicios

La API del Hito 2 es un **monolito modular**: una función HTTP desplegable llamada `api` contiene los módulos de inventario y lista de compras, separados internamente en rutas, controlador, servicio y repositorio. Ambos recursos comparten Firebase Authentication, la base Firestore y el mismo ciclo de publicación. Tener módulos separados dentro de una función no los convierte en microservicios.

Esta decisión se ajusta al alcance actual de FrescApp: dos recursos que trabajan sobre datos de las mismas cuentas y no tienen requisitos distintos de despliegue, carga o disponibilidad. Si inventario y lista de compras fueran servicios independientes, cada uno podría escalarse y publicarse por separado. También habría que operar más servicios y, cuando colaboraran, coordinar llamadas de red, autenticación, registros, monitoreo y consistencia de datos. Para este prototipo semestral, esos costos no responden a una necesidad actual.

El costo es compartir despliegue y escalado: un problema en la función `api` puede afectar a ambos recursos. Reconsideraríamos el diseño si uno necesitara desplegarse, escalarse o cumplir disponibilidad de forma independiente; separarlo requeriría además límites de datos y contratos claros.

4. Capas y recorrido de una petición

```
Cliente REST / Postman
| HTTPS + Firebase ID token
▼
Cloud Function HTTPS `api`
▼
Router · functions/api/routes.js
resuelve recurso, método y parámetro de ruta
▼
Controlador · functions/api/controller.js
verifica identidad, valida HTTP y forma respuestas
▼
Servicio de dominio · functions/api/service.js
aplica reglas de inventario y lista de compras
▼
Repositorio · functions/api/repository.js
consulta y modifica documentos del usuario en Firestore
▼
Cloud Firestore
```

Ejemplo `PATCH /v1/inventario/{id}`: el controlador verifica el token, rechaza cuentas anónimas o en eliminación y valida ruta/método; el servicio acepta solo campos modificables y valida la cantidad, ubicación y fecha; el repositorio actualiza únicamente `usuarios/{uid}/inventario/{id}`; finalmente la API devuelve el documento actualizado o un error HTTP con código y mensaje.

5. Contrato REST

Cloud Functions v2 permite exponer handlers HTTP sobre HTTPS y ejecutar el servicio bajo demanda (Firebase, s. f.-c; Firebase, s. f.-d). La URL base después del despliegue es `https://southamerica-west1-PROJECT_ID.cloudfunctions.net/api`. En el emulador local es `http://127.0.0.1:5001/demo-frescapp-hito2/southamerica-west1/api`.

Todas las rutas requieren `Authorization: Bearer <Firebase ID token>`. Las solicitudes con cuerpo requieren `Content-Type: application/json`. La identidad se toma del token, no de un parámetro proporcionado por el cliente.

Recurso	Método y ruta	Cuerpo / parámetros	Éxito
Inventario	`GET /v1/inventario?limit=50&cursor={id}`	`limit` opcional de 1 a 100; cursor opcional de paginación	`200`, `{ data: [...], pagination: { limit, nextCursor } }`
Inventario	`POST /v1/inventario`	Campos: `codigoBarras`, `nombre`, `marca`, `categoria`, `formato`, `unidad`, `cantidad`, `vencimiento`; opcionales `ubicacion`, `abiertoEn`	`201`, `{ data: {...} }` y cabecera `Location`
Inventario	`GET /v1/inventario/{id}`	Identificador del documento	`200`, `{ data: {...} }`
Inventario	`PATCH /v1/inventario/{id}`	Al menos uno de `cantidad`, `vencimiento`, `ubicacion`, `abiertoEn`	`200`, `{ data: {...} }`
Inventario	`DELETE /v1/inventario/{id}`	Identificador del documento	`204`, sin cuerpo
Lista de compras	`GET /v1/lista-compras?limit=50&cursor={id}`	`limit` opcional de 1 a 100; cursor opcional	`200`, `{ data: [...], pagination: { limit, nextCursor } }`
Lista de compras	`POST /v1/lista-compras`	`{ nombre, cantidad, unidad }`	`201`, `{ data: {...} }` y cabecera `Location`
Lista de compras	`GET /v1/lista-compras/{id}`	Identificador del documento	`200`, `{ data: {...} }`
Lista de compras	`PATCH /v1/lista-compras/{id}`	Uno o más de `nombre`, `cantidad`, `unidad`, `comprado`	`200`, `{ data: {...} }`
Lista de compras	`DELETE /v1/lista-compras/{id}`	Identificador del documento	`204`, sin cuerpo

Ejemplos

```
POST /v1/lista-compras
Authorization: Bearer <Firebase ID token>
Content-Type: application/json

{"nombre":"Arroz para la semana","cantidad":2,"unidad":"unidad"}
```

```
{"data":{"id":"abc123","nombre":"Arroz para la semana","cantidad":2,"unidad":"unidad","comprado":false}}
```

```
PATCH /v1/inventario/abc123
Authorization: Bearer <Firebase ID token>
Content-Type: application/json

{"cantidad":3,"ubicacion":"Despensa"}
```

Errores

Estado	Código	Cuándo
400	VALIDATION_ERROR, INVALID_ID, INVALID_LIMIT, INVALID_CURSOR	Cuerpo, ruta, límite o cursor inválido
401	UNAUTHENTICATED, INVALID_TOKEN	Falta un token válido o está vencido/revocado
403	ACCOUNT_REQUIRED, ACCOUNT_DELETION_PENDING	Cuenta anónima o cuenta bloqueada por eliminación
404	ROUTE_NOT_FOUND, NOT_FOUND	Ruta o recurso no encontrado
405	METHOD_NOT_ALLOWED	Método no admitido para esa ruta; incluye Allow
415	UNSUPPORTED_MEDIA_TYPE	Cuerpo no enviado como JSON
422	PRODUCT_IDENTITY_MISMATCH	El código y la identidad del producto no coinciden con el catálogo permitido
500	INTERNAL_ERROR	Falla inesperada; no expone detalles internos

Las respuestas de error usan `{ "error": { "code": "...", "message": "..." } }`. La versión forma parte de la ruta (`/v1`); cambios incompatibles deben introducir `/v2`.

6. Persistencia y credenciales

El detalle del modelo está en el modelo de datos. La API guarda datos en subcolecciones privadas `usuarios/{uid}/inventario` y `usuarios/{uid}/listaCompras`; el servidor asigna los identificadores y las marcas de tiempo. Como el Admin SDK no está limitado por las reglas de los SDK cliente, la API verifica identidad, deriva el UID del token y valida las entradas en el servidor.

Cloud Functions usa Application Default Credentials; en local, Functions Emulator conecta los servicios emulados (Firebase, s. f.-b). `functions/.env.example` documenta `ENFORCE_APP_CHECK` y `.env.example` contiene solo valores públicos de la app. Git ignora `.env.local`. No se distribuye un JSON de cuenta de servicio ni se guarda una clave privada en `EXPO_PUBLIC_*`.

7. Instalación y demostración local

Requisitos documentados del proyecto: Node.js 22 y Java 21 para Firebase CLI.

```
npm ci
npm ci --prefix functions
npx firebase emulators:start --only auth,firestore,functions --project demo-frescapp-hito2
```

Con los emuladores en ejecución, importa `FrescApp-Hito2.postman_collection.json` en Postman, asigna una contraseña sintética en la variable local `password` y envía primero `0. Crear cuenta local y guardar token`. La colección conserva el ID token y UID recibidos del emulador. En otra terminal prepara los datos sintéticos, sustituyendo el UID por el valor guardado:

```
$env:GCLOUD_PROJECT = 'demo-frescapp-hito2'
$env:FIRESTORE_EMULATOR_HOST = '127.0.0.1:8080'
node db/crear-estructura-hito2.cjs --uid UID_DE_PRUEBA
```

Luego ejecuta, en orden, las operaciones de inventario y lista de compras en Postman. La colección muestra el código de estado y el cuerpo devuelto; después de crear y actualizar, repetir `GET` muestra el dato persistido. El script de preparación rechaza cualquier destino que no sea un emulador local con ID de proyecto `demo-*`.

8. Conclusión

El back-end expone dos recursos REST con operaciones de creación, lectura, actualización y eliminación, conserva Firestore como persistencia y protege los registros con tokens de Firebase Authentication. La API y la colección Postman permiten preparar una demostración reproducible usando datos ficticios en emuladores. No se ha desplegado el servicio en el proyecto Firebase de producción; la evaluación se puede realizar primero en el entorno local descrito.

9. Referencias técnicas

- Firebase. (s. f.-a). *Add data to Cloud Firestore*. https://firebase.google.com/docs/firestore/manage-data/add-data
- Firebase. (s. f.-b). *Add the Firebase Admin SDK to your server*. https://firebase.google.com/docs/admin/setup
- Firebase. (s. f.-c). *Call functions via HTTP requests*. https://firebase.google.com/docs/functions/http-events
- Firebase. (s. f.-d). *Get started with Cloud Functions*. https://firebase.google.com/docs/functions/get-started
- Firebase. (s. f.-e). *Verify ID tokens*. https://firebase.google.com/docs/auth/admin/verify-id-tokens
- Express.js. (s. f.-a). *Routing*. https://expressjs.com/en/guide/routing/
- Express.js. (s. f.-b). *Using middleware*. https://expressjs.com/en/guide/using-middleware/
- Fastify. (s. f.). *Validation and serialization*. https://fastify.dev/docs/latest/Reference/Validation-and-Serialization/
- Google Cloud. (s. f.). *What is Cloud Run?* https://docs.cloud.google.com/run/docs/overview/what-is-cloud-run

Consulta: 4 de noviembre de 2026. Estas fuentes respaldan las decisiones de plataforma HTTP, framework, verificación de tokens y persistencia con Admin SDK.

Capas y credenciales locales

Límites claros entre HTTP, dominio y Firestore; configuración segura para repetir la prueba.

Responsabilidad de cada módulo

MÓDULO	RESPONSABILIDAD	LÍMITE
routes.js	Resuelve versión, recurso, ID, método y acción.	Sin consultas ni reglas de dominio.
controller.js	Verifica el ID token, procesa contenido/parámetros HTTP y forma status, cuerpo y cabeceras.	No accede directamente a Firestore.
service.js	Aplica política de cuenta y reglas de inventario/compra; coordina validadores y repositorios.	No recibe req/res ni construye HTTP.
validators.js	Valida campos, tipos, formatos y límites de entrada.	Sin I/O ni dependencias HTTP.
repository.js	Lee y escribe recursos privados, catálogo y estado de eliminación en Firestore.	No decide reglas de identidad ni status HTTP.

Dos archivos de ejemplo, dos ámbitos

CLIENTE EXPO / .env.example	FIREBASE FUNCTIONS / functions/.env.example
Contiene solo valores públicos EXPO_PUBLIC_*; no lleva contraseñas, tokens ni claves privadas. Es opcional para ejecutar únicamente el backend. Si se inicia la app, copiar a **.env.local**.	Contiene el parámetro no secreto **ENFORCE_APP_CHECK=false** para la prueba local. Copiar a **functions/.env**. El Admin SDK usa el emulador local; no se necesita un JSON de cuenta de servicio.

```
if (-not (Test-Path 'functions/.env')) { Copy-Item 'functions/.env.example' 'functions/.env' }
if (-not (Test-Path '.env.local')) { Copy-Item '.env.example' '.env.local' }  # solo si se ejecuta la app
```

Producción: usar Application Default Credentials y la identidad de servicio de Cloud Functions. No crear ni subir llaves JSON al repositorio. Git excluye los archivos .env reales, **.runtimeconfig.json** y archivos JSON de credenciales.